Technical Design — Ingestion Module

Theme & Epic Generation · Ingestion

1. Purpose & scope

The ingestion module converts **historical IDMT Engagement Request tickets** into a trusted corpus that the data-science (generation) module retrieves from. It produces two things for every eligible ticket:

1. a **condensed, durable record** of the ticket's content (system of record), and
2. **ground-truth labels** — the Value Streams, Stages, and L2/L3 capabilities a Business Architect actually assigned in Jira — for retrieval, evaluation, and prompt precedent.

Ingestion is **offline/batch** and contains no runtime generation logic. It writes to **Cosmos** (the system of record) and **idp_teg_data** (the retrieval index). The governed Value Stream / Stage / L2 / L3 catalogue is **not** produced here — it is the org's gold data in the **Azure SQL DB**, consumed as-is.

2. Flow

```
STAGE 0 — Ticket identification (Neo4j 5-filter funnel)
  Neo4j JIRA graph  →  one Cypher query (L2→L6 funnel)  →  list of usable IDMT ticket keys

STAGE 1 — Per-ticket ingestion (run once per identified key)
  IDMT Engagement Request (Jira)
    → fetch ER + linked Themes
    → attachment extraction + idea-card detection
    → raw text assembly (~24k-token budget)
    → Condense (LLM)              → summaryFields + rawText
    → ground-truth extraction     → Value Stream, Stages, L2/L3 (from Jira)
    → embed summaryFields
    → write Cosmos (SoR docs + GT) + upsert idp_teg_data (searchText + content_vector)
```

3. Ticket identification (which tickets we ingest)

A **distinct first stage, upstream of the per-ticket pipeline.** We do **not** ingest every IDMT ticket — we first identify the **usable Value-Stream cohort** and produce a list of their keys. The per-ticket pipeline (Stage 1) then runs once per identified key.

Identification runs against the **Neo4j JIRA graph** (env: `NEO4J_URI / USER / PASSWORD / DATABASE`) as a **single Cypher query** implementing a 5-filter funnel (L2→L6). A ticket survives to the cohort only if **all** hold:

filter	rule
L2 — is an IDMT Engagement Request, recent	`key` starts with `IDMT-`, `issueType = "Engagement Request"`, `creationDateEpoch ≥ since` (default **2023-01-01**)
L3 — not in a dead status	`status NOT IN {Cancelled, Blocked, New Request}`
L4 — is implemented by a linked issue	the IDMT has ≥1 **inbound "implemented by"** link — i.e. a Theme *implements* it. (From the IDMT's side this relationship is inward; the same link is outward "implements" on the Theme. We read it from the IDMT, so it appears inward.)
L5 — the linked issue is a Theme	the linked key resolves to a JIRA node with `issueType = "Theme"`
L6 — the Theme carries a Value Stream	the Theme's `businessValueStreams` matches `…{VSR\d+}` (a valid Value Stream id is present)

The query returns the **distinct ER keys** that pass all five; that key set is the cohort the batch ingestion run consumes.

> **Status note.** The ER-identification status exclusion is **{Cancelled, Blocked, New Request}** — *not* the same as the Epic-level skip in §7.2 (which excludes **Cancelled** only and keeps To Do). The funnel rejects whole tickets that were dropped, on hold, or never started; the Epic skip only drops cancelled stages within an otherwise-valid ticket.

Validity is also re-checked during ingestion: a linked Theme whose Business Value Stream does not resolve to an approved Value Stream is dropped, so only genuinely labelled tickets enter the corpus.

4. Source extraction — idea card & attachments

The business content comes from the ticket description and its attachments:

- **Idea-card first.** If an attachment named/tagged `idea_card.ppt` / `idea_card.pptx` exists, it is the primary business-context source.
- **Fallback.** If no idea card, use the Jira **description** plus the supported attachments — **up to 8** — in priority order **PowerPoint → PDF → Word**.
- **Extraction by format:** `.pdf` (PDFium), `.pptx` (python-pptx), `.docx` (python-docx). Legacy binary `.ppt` / `.doc` and image-only files yield no text and are skipped. No OCR.

5. Raw text assembly

The description and extracted attachment text are concatenated into a single **raw text** blob and **greedily packed into a ~24k-token budget** in source-priority order (idea card first, then description, then the remaining supported attachments). Highest-priority content is never displaced or truncated to fit a lower-priority attachment; the token budget is the only cap.

6. Condense (LLM)

A single LLM pass extracts structured context from the raw text, so downstream steps don't re-process it. **Output:** `summaryFields` —

field	meaning
`generatedSummary`	LLM summary of the ticket
`businessProblem`	the pain point / problem statement
`businessCapability`	the desired capability / outcome
`keyTerms`	domain terms & acronyms
`stakeholders`	stakeholder groups
`systemsAndProducts`	referenced systems, platforms, products

The step also carries through `rawText` (the ~24k-token consolidated content). `summaryFields` are used for retrieval/routing; `rawText` is stored for the generation module to read at runtime.

7. Ground-truth extraction (from Jira)

Ground truth is the Business Architect's recorded answer — **read directly from Jira fields, not inferred.**

7.1 Value Stream

Read **directly from each linked Theme's "Business Value Stream" field**, formatted `<name> {id}` (e.g. `Configure Price {VSR00074590}`). The name is taken **as-is** — **no fuzzy matching, no LLM confirmation, no catalogue re-resolution.** The field id is discovered by name once and cached.

7.2 Themes → Epics (the parent–child lookup)

A Theme (GROUP issue) is connected to its Epics through the Jira **parent–child relationship**, not issue links — so child Epics do **not** appear in the Theme's inward/outward links. We therefore do a **reverse lookup**: find the Epics whose parent is the Theme.

```
parent = "GROUP-23618" AND issuetype = Epic
"Parent Link" = "GROUP-23618" AND issuetype = Epic
```

The two JQL paths (standard `parent` and the `Parent Link` custom field) plus any implement/Epic issue-links on the Theme are **unioned and de-duplicated by key**. Epics in status **Cancelled** are excluded (To Do is kept — a planned-but-not-started stage is still valid GT).

7.3 Stage

For each Epic, the stage is read from its **Value Stream Stage** cascading field (the canonical stage id + name; `match_method = field`). Fuzzy matching of the Epic **summary** against the catalogue is a **fallback used only when that field is absent**. Stages whose id is **not in the governed catalogue** (retired / out-of-catalogue) are **dropped** from ground truth — the model can't be graded on options that don't exist.

7.4 L2 / L3 capabilities & Business Needs

Read from the Theme's fields: **L2 Business Capability Model**, **L3 Business Capability Model**, and **Business Needs**. These are theme-level ground truth (recorded on the live GROUP issue), not per stage.

8. Storage

8.1 Cosmos — system of record

Durable source lineage + ground truth. Two document types:

IDMT / Engagement Request document — top-level: `id` (uuid), `key` (IDMT-####), `sourceId` (7-digit stable id), `source`, `entityType`, `domain: "WORKITEM"`, `createdAt/By`, `lastModifiedAt/By` (ingestion lifecycle), `parentRef` (null). `properties`: `description`, `summary` (ticket title), `businessSummary` (LLM summary), `creationDate` / `insightsTime` (source ticket dates), `keyTerms`, `businessProblem`, `businessCapability`, `stakeholders`, `systemsAndProducts`, `rawText`.

> Themes are **separate documents** (found via `parentRef`), **not** embedded as a `themes[]` array on the IDMT doc. No `generationSignals` are stored.

Theme document — top-level: `id` (uuid), `key` (GROUP-####), `sourceId` (7-digit), `source`, `entityType`, `domain: "WORKITEM"`, `createdAt/By`, `lastModifiedAt/By` (ingestion lifecycle), `parentRef` (the parent IDMT's 7-digit id). `properties`: `summary` (Theme title), `description`, `valueStream` (string `"<Name> {vs_id}"`), `creationDate` / `insightsTime` (Theme dates).

8.2 idp_teg_data — retrieval index (retrieval-only)

Holds **only historical Engagement-Request documents**. Per document: `key`, `sourceId`, `entityType`, `status`, `searchText`, `content_vector`. It returns ranked ids; the corresponding ticket details are enriched from Cosmos at query time. **Value Streams are not indexed.**

8.3 Azure SQL DB — governed catalogue (consumed, not produced)

The Value Stream, Stage, and L2/L3 capability catalogue (the 50 approved Value Streams with their stages and capabilities) is the org's **gold data in Azure SQL**, read as-is at query time by `valueStreamId`. Ingestion does **not** define or store this catalogue.

9. Embedding & retrieval text

The `summaryFields` are assembled into a single retrieval text and embedded into `content_vector`; this is what a live query embeds against to find similar past tickets. The same retrieval-text shape is used for a stored ticket and a live query, so they land in the same vector space.

10. Jira field reference

field	location	id
Business Value Stream	Theme	discovered by name (cached)
Value Stream Stage	Epic	`customfield_18700`
Business Needs	Theme	`customfield_20900`
L2 Business Capability Model	Theme	`customfield_18602`
L3 Business Capability Model	Theme	`customfield_18603`
Parent Link	Epic→Theme	`customfield_11401`

Skipped status: **Cancelled** (`cancelled` / `canceled`). **To Do is kept.**

11. Rules & conventions

- **Ground truth is read, never inferred** — VS from the Business Value Stream field, stages from the Value Stream Stage field; fuzzy/summary matching is a stage-only fallback.
- **Uncatalogued (retired) stages are dropped** from ground truth.
- **The index is retrieval-only** — `searchText` + `content_vector`; no labels, themes, or signals stored in it.
- **The Value Stream catalogue is Azure SQL gold data**, consumed as-is — not redefined or stored here.
- **Unit tests make no live Jira / Azure / LLM calls** — clients are injected (fakes).